

AgentTrust OS – Bharat Academix

CodeQuest 2026

Runtime Security & Governance Platform for Enterprise AI Agents

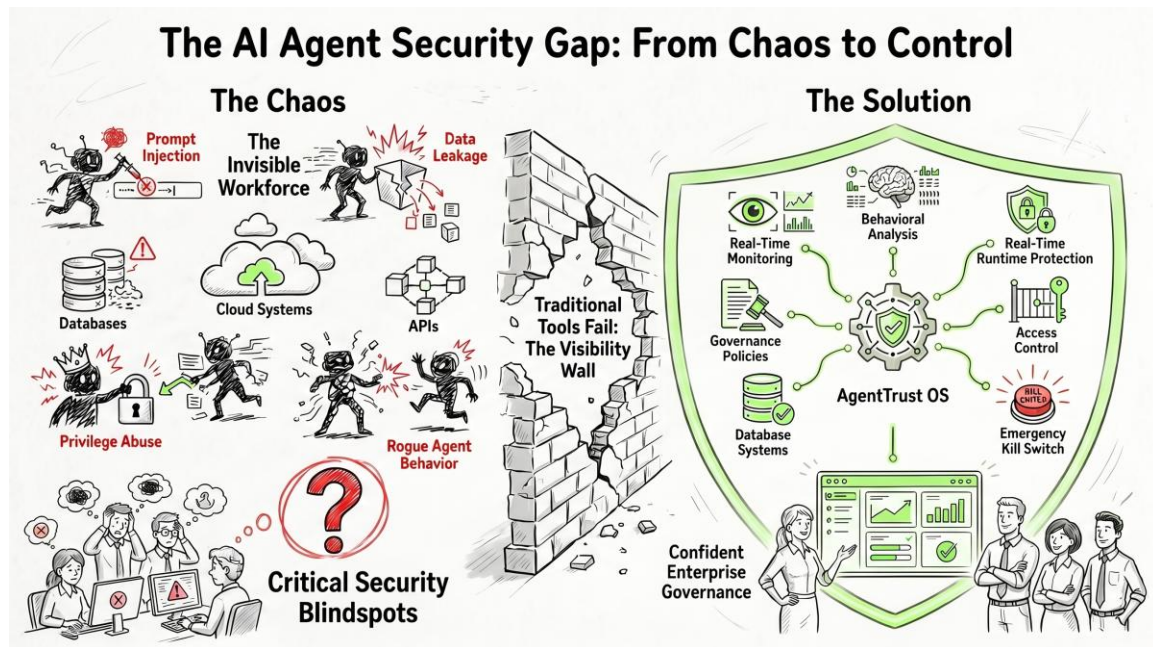
Prepared for Bharat Academix CodeQuest 2026 Round-2 Submission

1. Problem & User Flow

Organizations are rapidly adopting AI agents such as Microsoft Copilot, OpenAI Agents, GitHub Copilot, MCP-based systems, and internal automation agents. However, they lack visibility and control over what these agents access, what actions they perform, and whether their behavior is safe.

AgentTrust OS acts as a real-time governance layer that monitors, evaluates, and prevents risky agent actions before they execute.

Problem Statement Visual

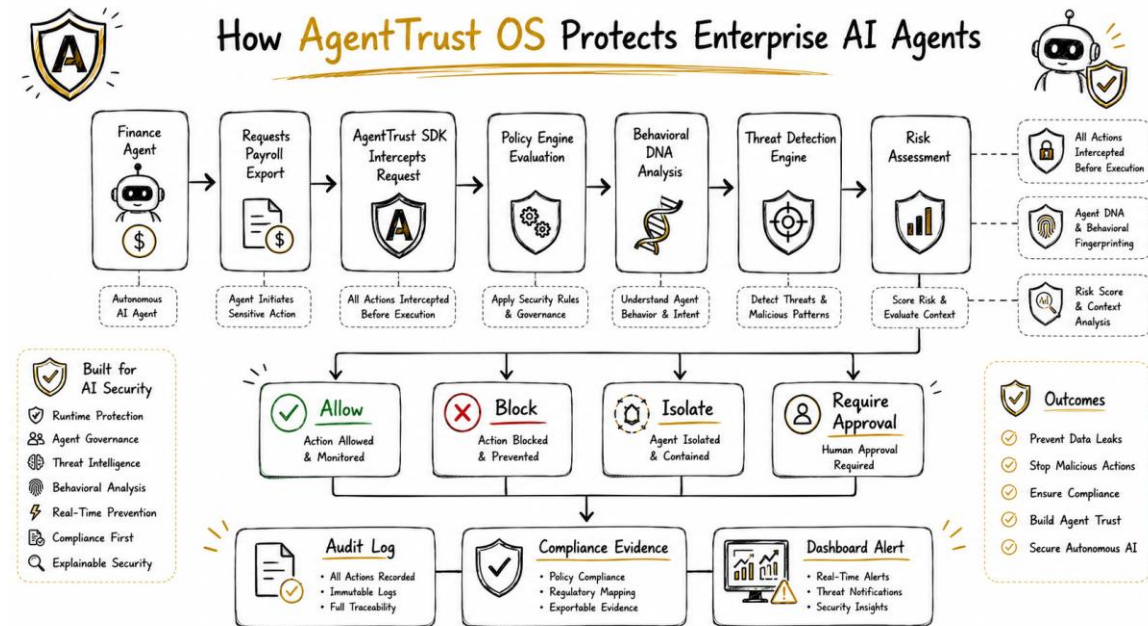


User Flow:

1. Agent performs an action.
2. AgentTrust SDK intercepts the request.
3. Policy Engine evaluates the action.
4. Behavioral Engine checks Agent DNA.

5. Threat Engine detects risks.
6. Prevention Engine decides Allow / Block / Isolate / Approval.
7. Dashboard updates in real time.

User Flow Diagram



2. Under-the-Hood Design

AgentTrust OS is built as a runtime security control plane.

Input Flow:

Agent -> SDK -> Event Pipeline -> Policy Engine + Behavior Engine + Threat Engine -> Prevention Engine -> Dashboard & Audit Logs

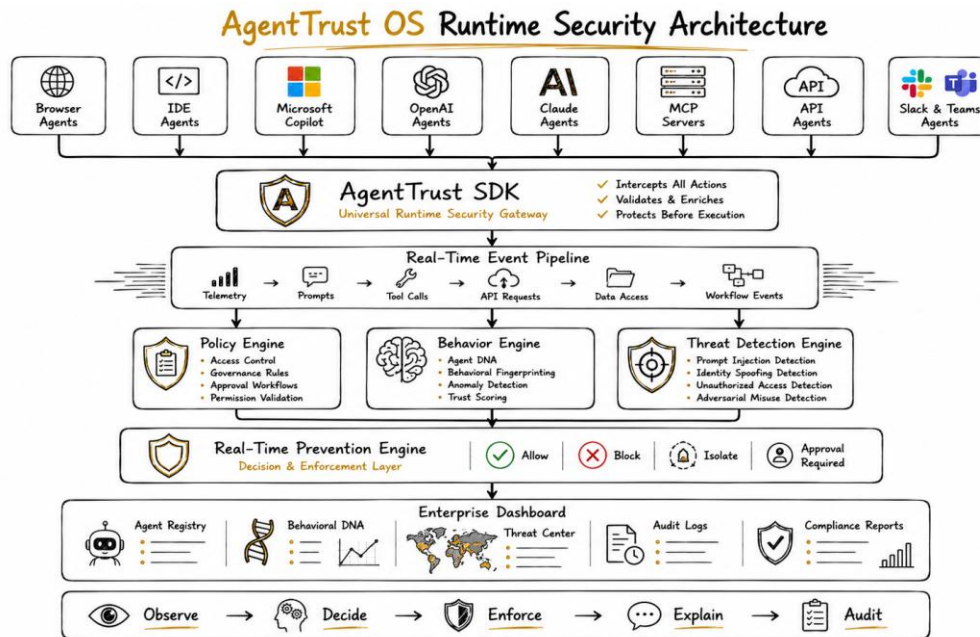
Key Components:

- Agent Registry
- Behavioral DNA Engine
- Policy Engine
- Threat Detection Engine
- Honeypot Engine
- Approval Workflow
- Audit Logging System

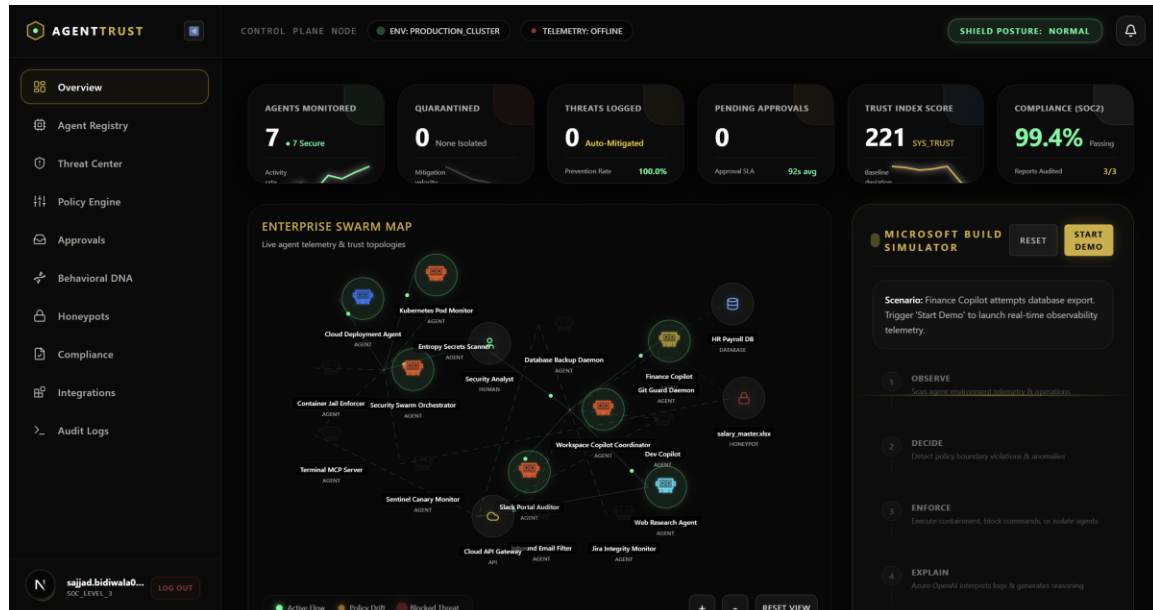
Risk Scoring:

Trust scores are dynamically updated based on agent behavior, policy violations, and honeypot interactions.

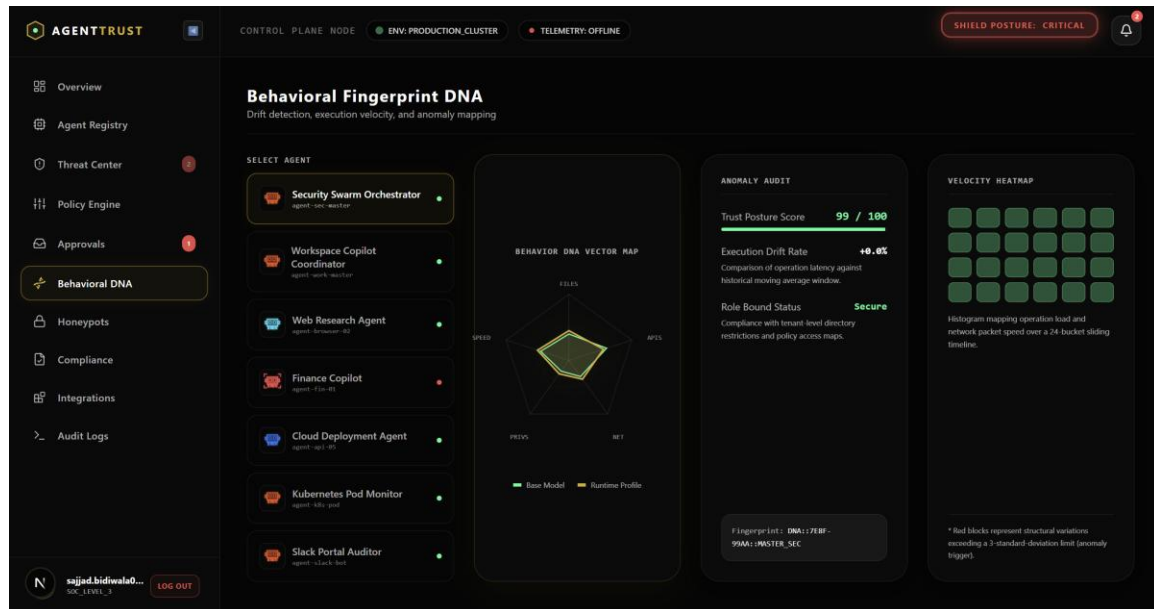
Technical flow architecture



Dashboard Overview Screenshot



Behavioral DNA Screenshot



3. Data Sources & References

Data used by AgentTrust OS:

- Agent telemetry events
- Supabase PostgreSQL database
- Security policies
- Honeypot assets
- Audit logs
- Approval workflow records
- API key metadata

Frameworks & Services:

- Next.js
- FastAPI
- Supabase
- WebSockets
- Azure OpenAI (currently mocked)
- Python SDK
- TypeScript SDK

4. Value Beyond a Generic LLM

A generic LLM can answer questions but cannot monitor and govern running enterprise agents.

Why AgentTrust OS is different:

- Intercepts actions before execution
- Maintains trust scores
- Detects behavioral drift
- Blocks risky operations
- Supports approvals
- Generates audit trails
- Detects honeypot access

Comparison:

ChatGPT:

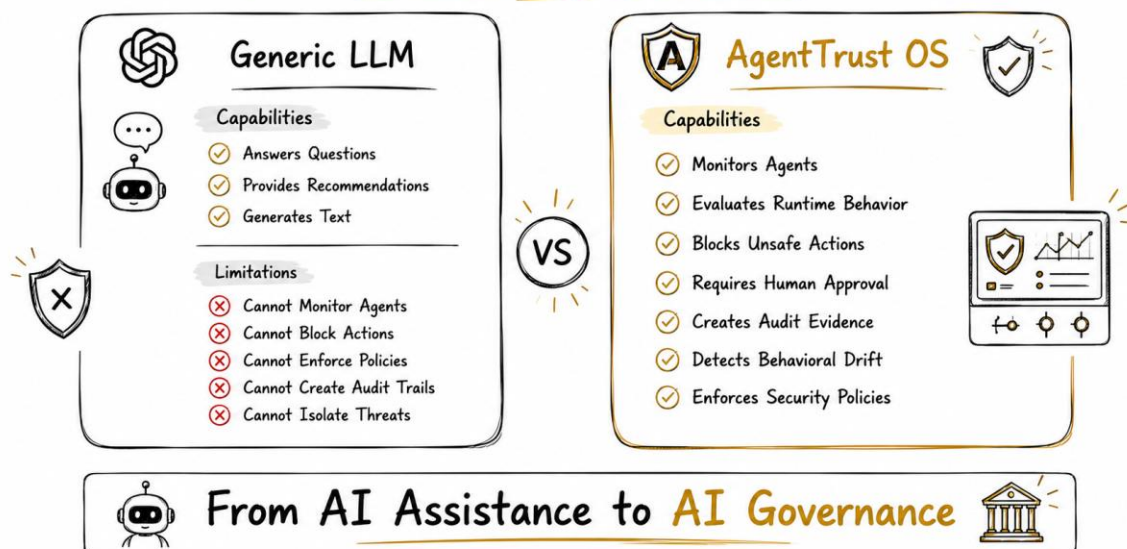
- Gives recommendations
- Cannot stop actions
- Cannot monitor enterprise agents

AgentTrust OS:

- Monitors agents
- Evaluates behavior
- Prevents actions
- Creates compliance evidence

ChatGPT vs AgentTrust comparison

Why AgentTrust OS Cannot Be Replaced by ChatGPT



- WebSockets

Database:

- Supabase PostgreSQL

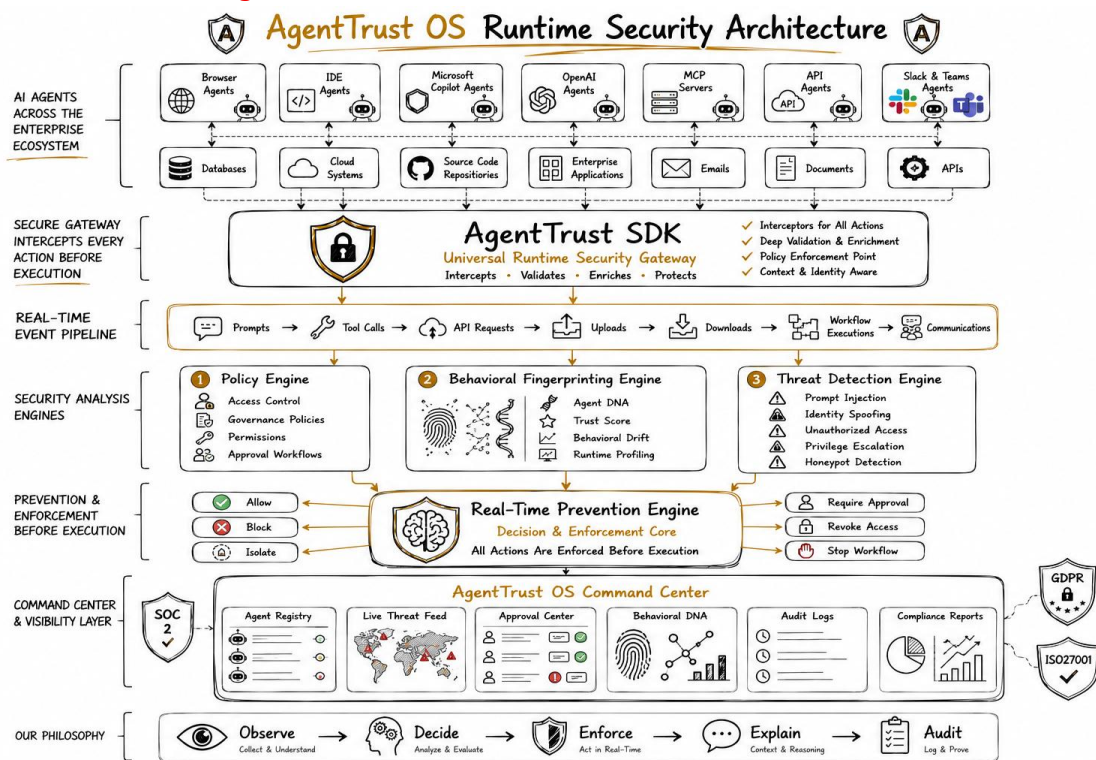
Deployment:

- Vercel
- Railway / Azure Container Apps

Security Layer:

- Policy Engine
- Behavioral Fingerprinting
- Threat Detection
- Honeypots
- Agent Isolation

Full architecture diagram



6. Demo Scenario & Limitations

Demo Scenario:

Finance Agent receives:

"Export employee salary database"

Processing:

1. Agent requests payroll data.
2. SDK captures event.
3. Policy Engine evaluates request.
4. Behavioral Engine detects anomaly.
5. Honeypot file is accessed.
6. Threat generated.
7. Agent isolated.
8. Approval request created.
9. Dashboard updates live.

Output:

- Threat detected
- Trust score reduced
- Agent isolated
- Audit record created

Threat Center Screenshot

The screenshot displays the Threat Intelligence Center dashboard. The top navigation bar includes the AGENTTRUST logo, environment controls (ENV: PRODUCTION_CLUSTER, TELEMETRY: OFFLINE), and a SHIELD POSTURE: CRITICAL indicator. The left sidebar lists navigation options: Overview, Agent Registry, Threat Center (active), Policy Engine, Approvals, Behavioral DNA, Honeypots, Compliance, Integrations, and Audit Logs. The main content area is titled 'Threat Intelligence Center' and shows two threat events. The first event, 'Critical: Honeypot Decoy Asset Accessed', includes a table with columns for Target Entity, Mitigation Applied, Detection Layer, and Status. The second event, 'Policy Violation: Unauthorized HR DB Query', also includes a similar table. A 'SECURITY EVENT SUMMARY' section on the right provides a count of alerts, and a 'CONTAINMENT STATUS' section shows the mitigation latency.

TARGET ENTITY	MITIGATION APPLIED	DETECTION LAYER	STATUS
Finance Copilot	Runtime Process Quarantine & Sandbox Isolation	Decoy Honeypot Engine	Contained

TARGET ENTITY	MITIGATION APPLIED	DETECTION LAYER	STATUS
Finance Copilot	Flagged & Logged	Policy Enforcement Engine	Pending Review

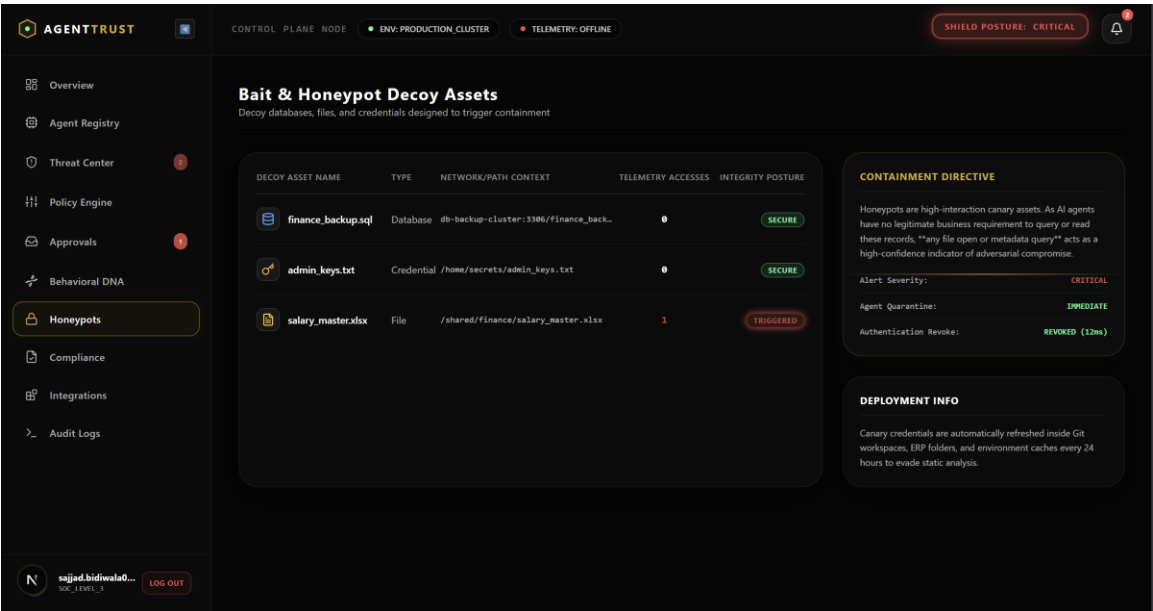
SECURITY EVENT SUMMARY	Count
Critical Alerts	1
High Alerts	0
Medium Alerts	1
Auto-Contained Events	1

CONTAINMENT STATUS

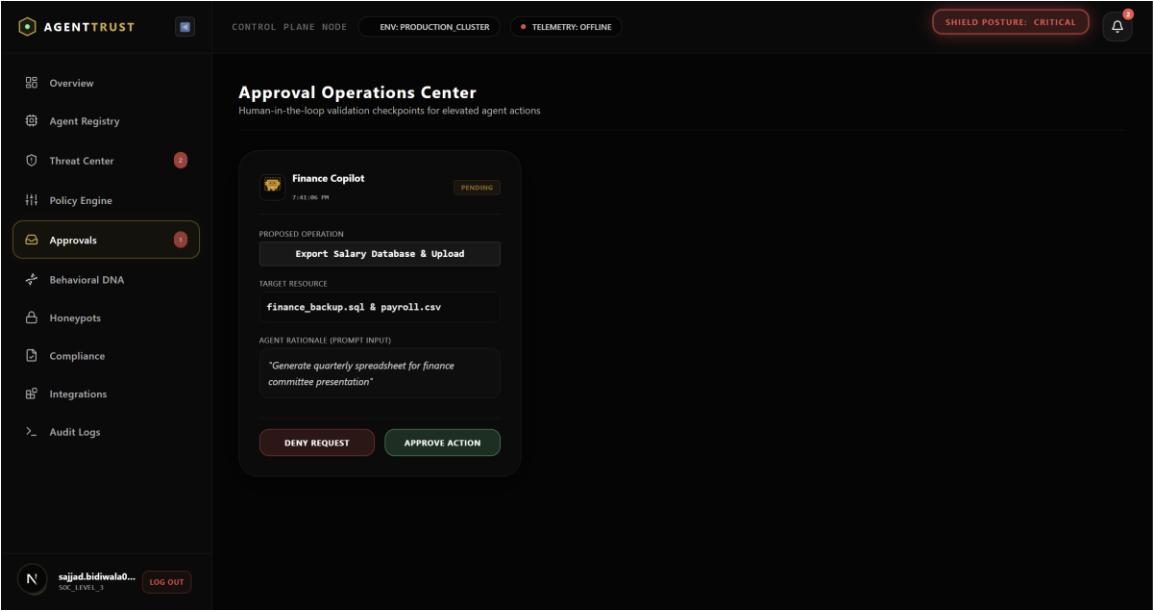
Mitigation engine operates in "Auto-Enforcement mode". All critical severity alerts trigger immediate process sandbox isolation within "12ms" of threat signature confirmation.

Mitigation Latency: 12 ms (Avg)

Honeypot Detection Screenshot



Approval Workflow Screenshot



Current Status:

Fully Working:

- Multi-tenancy
- Agent Registry

- WebSockets
- Behavioral DNA
- Honeypots
- Approval Workflow
- Agent Isolation
- SDKs

Partially Complete:

- Explainability Engine

Mocked:

- Azure OpenAI Integration

Planned:

- Chrome Extension
- VS Code Extension
- Native MCP Integrations

7. GitHub, Demo & Future Roadmap

GitHub Repository:

<https://github.com/Sajjad0106/AgentTrust-OS>

Demo Link:

<https://agent-trust-os-dashboard.vercel.app/>

Roadmap:

Phase 1: Current MVP

Phase 2: Browser & IDE Connectors

Phase 3: Enterprise Integrations

Phase 4: Industry Standard Trust Layer for AI Agents

Conclusion:

AgentTrust OS demonstrates a working runtime security platform that helps organizations safely deploy AI agents through monitoring, behavioral analysis, prevention, explainability, and governance.

